coreclaw-api-spec

CoreClaw API 一覧・API仕様書

本書は、CoreClaw の現行コードに存在する HTTP API と WebSocket メッセージを整理した仕様書である。

1. 共通仕様

- ベースパスは /api とする。
- レスポンスは原則 JSON とする。
- 正常系 HTTP ステータスは 200、201、202 を利用する。
- 異常系では 400、404、409、500、502、503 などを返す。
- 認証は独立したユーザー認証機構ではなく、ローカル利用前提のアプリ設定および GitHub Token を前提とする。

2. 実験 API

2.1 実験一覧取得

- Method: GET
- Path: /api/experiments
- Query: status 任意
- Response: 実験配列

2.2 実験作成

- Method: POST
- Path: /api/experiments
- Request Body

```
{
  "name": "Experiment A",
  "description": "optional",
  "sync_repo": "owner/repo",
  "skill": "scientist",
  "mcp_servers": "[]"
}
```

- Response: 作成済み実験オブジェクト

2.3 実験詳細取得

- Method: GET
- Path: /api/experiments/:id
- Response: 実験オブジェクト

2.4 実験更新

- Method: PATCH
- Path: /api/experiments/:id
- Request Body: 更新対象フィールド
- Response: 更新済み実験オブジェクト

2.5 実験削除

- Method: DELETE
- Path: /api/experiments/:id
- Response

```
{ "ok": true }
```

3. メッセージ API

3.1 メッセージ一覧取得

- Method: GET
- Path: /api/experiments/:id/messages
- Query: limit, offset
- Response

```
{
  "messages": [],
  "total": 0
}
```

3.2 メッセージ投稿

- Method: POST
- Path: /api/experiments/:id/messages
- Request Body

```
{ "content": "ユーザー入力" }
```

- Response: 保存済みユーザーメッセージ

3.3 メッセージ検索

- Method: GET
- Path: /api/experiments/:id/messages/search
- Query: q
- Response

```
{ "messages": [] }
```

4. タスク履歴・活動ログ API

4.1 プロセス履歴取得

- Method: GET
- Path: /api/experiments/:id/process-history
- Response

```
{ "tasks": [] }
```

4.2 活動ログ取得

- Method: GET
- Path: /api/experiments/:id/activity
- Query: limit

- Response

```
{ "events": [] }
```

5. ファイルアップロード・成果物 API

5.1 ファイルアップロード

- Method: POST
- Path: /api/experiments/:id/upload
- Content-Type: multipart/form-data
- Response

```
{ "uploaded": ["file1.txt"] }
```

5.2 アップロード一覧取得

- Method: GET
- Path: /api/experiments/:id/uploads
- Response: ファイル名配列

5.3 成果物ツリー取得

- Method: GET
- Path: /api/experiments/:id/artifacts/tree
- Response: path と type を持つ配列

5.4 成果物一覧取得

- Method: GET
- Path: /api/experiments/:id/artifacts
- Response: 成果物パス配列

5.5 ZIP インポート

- Method: POST
- Path: /api/experiments/:id/import-zip
- Content-Type: multipart/form-data または raw ZIP

- Response

```
{ "ok": true, "imported": 12 }
```

5.6 成果物一括ダウンロード

- Method: GET
- Path: /api/experiments/:id/download
- Response: application/zip

5.7 個別成果物取得

- Method: GET
- Path: /api/experiments/:id/artifacts/:path
- Query: download=1 任意
- Response: 対象 MIME のファイルストリーム

6. メモリ API

6.1 メモリ取得

- Method: GET
- Path: /api/experiments/:id/memory
- Response

```
{
  "memory": {
    "experiment_id": "...",
    "summary": "...",
    "summarized_count": 0
  },
  "total_messages": 0,
  "is_summarizing": false
}
```

6.2 メモリ消去

- Method: DELETE
- Path: /api/experiments/:id/memory
- Response

```
{ "ok": true }
```

6.3 メモリ要約開始

- Method: POST
- Path: /api/experiments/:id/memory/summarize
- Response

```
{ "ok": true, "message": "Summarisation started" }
```

7. GitHub 同期 API

7.1 実験 Push

- Method: POST
- Path: /api/experiments/:id/sync
- Response: ok, message 等を含む同期結果

7.2 実験 Pull

- Method: POST
- Path: /api/experiments/:id/pull
- Response: ok, message 等を含む同期結果

8. バージョン・更新 API

8.1 バージョン取得

- Method: GET
- Path: /api/versions
- Response: コンポーネントごとのバージョン情報

8.2 Copilot 認証状態取得

- Method: GET
- Path: /api/auth/copilot-status
- Query: refresh=1 任意

- Response

```
{
  "ok": true,
  "state": "authenticated",
  "source": "settings",
  "message": "...",
  "checkedAt": "..."
}
```

8.3 更新確認

- Method: POST
- Path: /api/check/:component
- Response: 更新有無や対象バージョン情報

8.4 更新実行

- Method: POST
- Path: /api/update/:component
- Response: 実行結果

8.5 サーバー停止

- Method: POST
- Path: /api/shutdown
- Response

```
{ "ok": true, "message": "Shutting down CoreClaw" }
```

9. Skills API

9.1 スキル一覧取得

- Method: GET
- Path: /api/skills
- Response: name、description、version、fileCount、Marketplace 状態などを含む配列

9.2 Marketplace 由来状態取得

- Method: GET
- Path: /api/skills/:name/marketplace-status
- Response: marketplaceImported、currentVersion、latestVersion、updateAvailable 等

9.3 Marketplace 一覧取得

- Method: GET
- Path: /api/skills/marketplace
- Response

```
{
  "groups": [],
  "sources": []
}
```

9.4 Marketplace インポート

- Method: POST
- Path: /api/skills/marketplace/import
- Request Body

```
{
  "group": "group-slug",
  "sourceId": "official"
}
```

- Response: インポート結果

9.5 スキルファイル一覧取得

- Method: GET
- Path: /api/skills/:name/files
- Response

```
{
  "name": "skill-name",
  "files": []
}
```

9.6 スキル作成

- Method: POST
- Path: /api/skills
- Request Body

```
{
  "name": "my-skill",
  "description": "desc",
  "content": "markdown content"
}
```

- Response: 作成済みスキル情報

9.7 スキル更新

- Method: PUT
- Path: /api/skills/:name
- Request Body: ZIP バイナリまたは SKILL.md テキスト
- Response

```
{ "name": "my-skill", "updated": true }
```

9.8 スキル削除

- Method: DELETE
- Path: /api/skills/:name
- Response

```
{ "name": "my-skill", "deleted": true }
```

9.9 スキルスキャン

- Method: POST
- Path: /api/skills/:name/scan
- Response

```
{
  "name": "my-skill",
  "status": "green",
  "filesScanned": 10,
```

```
    "findings": [],
    "highCount": 0,
    "medCount": 0,
    "codeBlockSkipped": 0,
    "whitelistedCount": 0
  }
```

9.10 スキャンホワイトリスト更新

- Method: PUT
- Path: /api/skills/:name/scan/whitelist
- Request Body

```
{ "keys": ["file:label"] }
```

- Response: whitelistedCount を含む結果

10. 設定 API

10.1 設定取得

- Method: GET
- Path: /api/settings
- Response: マスク済み設定オブジェクト

主なキー:

- github_token
- copilot_model
- github_mcp_tools
- my_skills_repo_url
- output_language
- ai_provider
- openai_api_key
- azure_openai_api_key
- azure_openai_endpoint
- ollama_url
- ollama_model
- github_username

- mcp_servers

10.2 設定保存

- Method: PUT
- Path: /api/settings
- Request Body: 上記設定キーを含むオブジェクト
- Response

```
{ "ok": true }
```

11. WebSocket 仕様

11.1 目的

- 実行中タスクとストリーミング応答をリアルタイム配信する。

11.2 クライアント送信メッセージ

タスク停止

```
{ "type": "stop", "taskId": "task-..." }
```

タスク一覧要求

```
{ "type": "list_tasks" }
```

11.3 サーバー送信メッセージ

実行タスク一覧

```
{ "type": "tasks", "tasks": [] }
```

タスク開始

```
{ "experimentId": "...", "type": "agent_start", "taskId": "..." }
```

出力チャンク

```
{ "experimentId": "...", "type": "agent_chunk", "taskId": "...", "chunk": "..." }
```

状態更新

```
{ "experimentId": "...", "type": "agent_status", "taskId": "...", "status": "..." }
```

完了通知

```
{ "experimentId": "...", "type": "agent_done", "taskId": "...", "message": {} }
```

失敗通知

```
{ "experimentId": "...", "type": "agent_error", "taskId": "...", "error": "..." }
```

メモリ更新

```
{ "experimentId": "...", "type": "memory_update", "memory": {} }
```

12. 異常系の基本方針

- 対象実験やスキルが存在しない場合は 404 を返す。
- 入力不足や形式不備は 400 を返す。
- 重複実行中のメモリ要約など競合状態は 409 を返す。
- 機能未設定や利用不可状態は 503 を返す。
- 外部サービスや Marketplace 取得失敗は 500 または 502 を返す。